

Planejamento de Movimento de Robôs

Trabalho Prático 2: A*, GVD e RRT com simulação no CoppeliaSim

Gabriel Vaz Cançado Ferreira
Phillip Ribeiro Costa

Universidade Federal de Minas Gerais

2026



Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

Introdução

Problema

Dado um ambiente com obstáculos, um ponto inicial e um objetivo, encontrar uma trajetória **livre de colisões** até o destino.

Três paradigmas clássicos foram implementados e comparados:

- **A*** — busca heurística em grafo (caminho mais curto);
- **Brushfire + GVD** — caminho centrado, de máxima folga;
- **RRT** — planejamento probabilístico por amostragem.

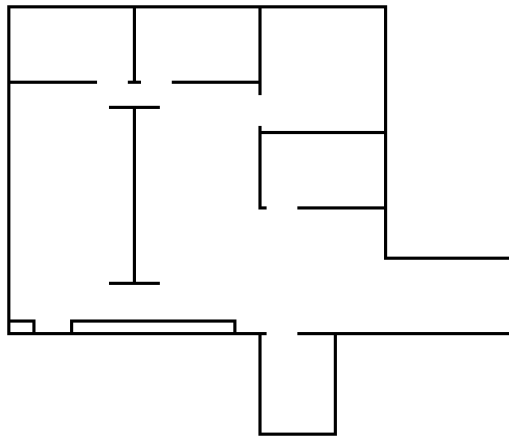
Tudo integrado ao simulador **CoppeliaSim**, com geração automática do ambiente 3D a partir dos mapas e execução das trajetórias.

Sumário

- 1 Introdução
- 2 Representação do Ambiente**
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

Representação do Ambiente

- **Occupancy grid**: cada célula é livre (0) ou obstáculo (1).
- Mapas carregados de imagens **PNG** (pixel escuro = obstáculo).
- **Inflação** dos obstáculos pelo raio do robô (morfológica): o robô vira um **ponto** e o caminho já respeita a margem de segurança.
- Conectividade em **8 direções**: custo 1 (reto) e $\sqrt{2}$ (diagonal).



Mapa *autolab*.

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A***
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

A*: busca heurística

Função de avaliação

$$f(n) = g(n) + h(n)$$

$g(n)$: custo acumulado do início até n ; $h(n)$: estimativa do custo restante até o objetivo.

- Heurística **Octile** (8-conexão): admissível e consistente $\Rightarrow$ caminho **ótimo** na discretização.
- Fila de prioridade (*heap*) + *closed set*.
- **Suavização** *line-of-sight* ao final remove o zigue-zague das diagonais.

A*: trecho central

```
def astar(gmap, start, goal, heuristic="octile"):
    h = HEURISTICS[heuristic]
    open_heap = [(h(start, goal), 0.0, start)] # (f, g, celula)
    came_from = {start: None}; g_score = {start: 0.0}; closed = set()
    while open_heap:
        _, g, current = heapq.heappop(open_heap)
        if current in closed: continue
        closed.add(current)
        if current == goal: return reconstruir(came_from, goal)
        for nb, step in gmap.neighbors(current): # 8-conexao, 1 e raiz(2)
            tentative = g + step
            if tentative < g_score.get(nb, float("inf")):
                g_score[nb] = tentative; came_from[nb] = current
                heapq.heappush(open_heap,
                              (tentative + h(nb, goal), tentative, nb))
```

Limitação: custo cresce com a resolução do grid.

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD**
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

Brushfire + GVD: caminho seguro

- Motivação: o A^* é curto, mas passa **colado** nas quinas. O GVD privilegia a **segurança** (máxima folga).
- Obstáculos rotulados em **componentes conexas**.
- **Brushfire exato** (propagação tipo Dijkstra): cada célula livre recebe a distância **euclidiana** ao ponto-base do obstáculo mais próximo.
- **GVD** = células equidistantes a dois obstáculos (eixo medial).
- Navegação em 3 trechos: entra no GVD $\rightarrow$ Dijkstra no esqueleto $\rightarrow$ sai para o objetivo.

GVD: trecho central

```
# Brushfire exato: propaga o ponto-base, mede distancia euclidiana real.
while heap:
    d, r, c = heapq.heappop(heap)
    if d > dist[r, c]: continue
    bri, bci = basis_r[r, c], basis_c[r, c] # ponto-base herdado
    for dr, dc in NB8:
        nr, nc = r + dr, c + dc
        if livre(nr, nc):
            nd = hypot(nr - bri, nc - bci)
            if nd < dist[nr, nc]:
                dist[nr, nc] = nd
                basis_r[nr, nc], basis_c[nr, nc] = bri, bci
                heapq.heappush(heap, (nd, nr, nc))
# GVD: vizinho com ponto-base distante (eixo medial) ou de outro obstaculo
for (r, c) in celulas_livres:
    for (nr, nc) in vizinhos8(r, c):
        gap2 = (basis_r[r,c]-basis_r[nr,nc])**2 + (basis_c[r,c]-basis_c[nr,nc])**2
        if owner[nr, nc] != owner[r, c] or gap2 >= LIMIAR:
            gvd.add((r, c)); break
```

Limitação: o eixo medial pode **fragmentar** em passagens estreitas.

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT**
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão

RRT: amostragem em espaço contínuo

- Constrói uma **árvore** no espaço contínuo (não preso ao grid).
- A cada iteração: amostra ponto aleatório $\rightarrow$ acha o nó mais próximo $\rightarrow$ *steering* (passo fixo) $\rightarrow$ checa colisão do segmento.
- **Goal bias**: parte das amostras é o próprio objetivo (acelera a convergência).
- **Suavização** *line-of-sight* ao final.

Limitação: probabilístico (não ótimo) e sofre com corredores estreitos (*narrow passage problem*).

RRT: trecho central

```
nodes = [start]; parent = [-1]
for _ in range(max_iter):
    # com prob. goal_bias, amostra o proprio objetivo
    amostra = goal if rng.random() < goal_bias else amostrar_livre()
    i = mais_proximo(nodes, amostra) # vizinho mais proximo
    novo = steer(nodes[i], amostra, passo) # avanca 'passo'
    if not segmento_livre(nodes[i], novo): continue # colisao
    nodes.append(novo); parent.append(i)
    if dist(novo, goal) <= tol and segmento_livre(novo, goal):
        nodes.append(goal); parent.append(len(nodes) - 2)
        return reconstruir(nodes, parent) # caminho encontrado
```

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim**
- 7 Resultados
- 8 Conclusão

Integração com o CoppeliaSim

- Conexão via **ZMQ Remote API**.
- Cena 3D gerada **automaticamente** do occupancy grid: piso + paredes (obstáculos agrupados em poucos retângulos).
- Robô = cilindro com marcador frontal; segue o caminho por um **controlador cinemático** (interpola posição e orientação).
- Cada execução tem **duas fases**:
 - 1 construção do algoritmo (explorados do A*, esqueleto do GVD, árvore do RRT);
 - 2 o robô percorre o caminho planejado.

Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados**
- 8 Conclusão

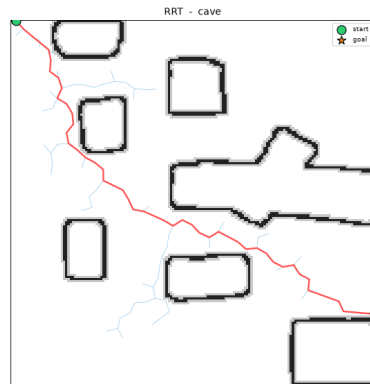
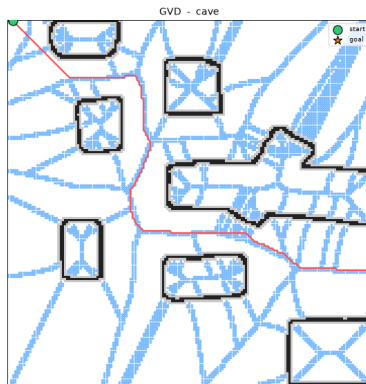
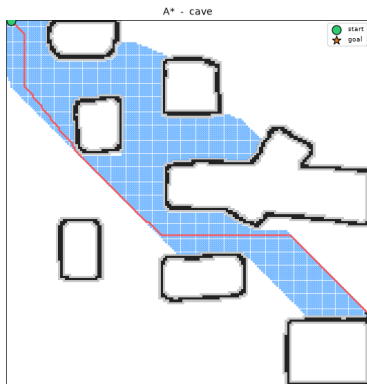
Resultados: comprimento dos caminhos

Mapa	A* (len)	GVD (len)	RRT (len)
autolab	200	220	221
cave	237	278	249
paredes	209	233	225
square_maze	434	485	536
circular_maze	568	625	falha*
icex-terceiro	108	falha**	117

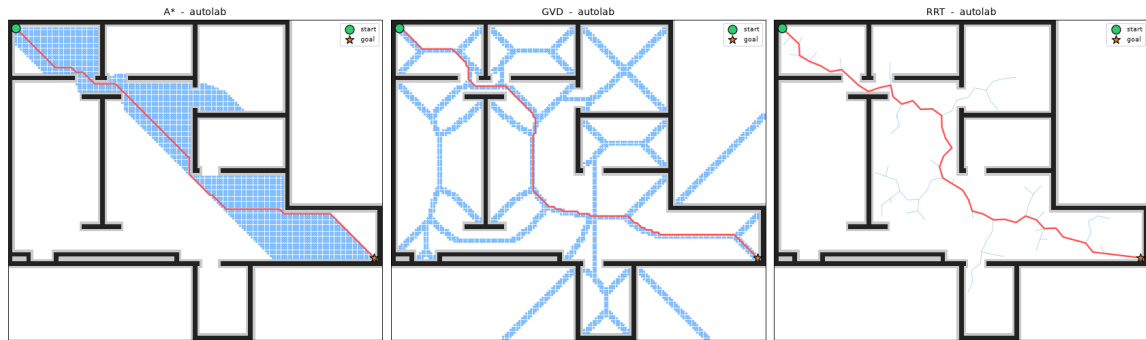
* RRT não resolve labirintos densos (*narrow passage*).

** GVD: o eixo medial fragmenta em portas estreitas; a rede principal não conecta *start* e *goal* (o A* resolve).

Resultados — cave

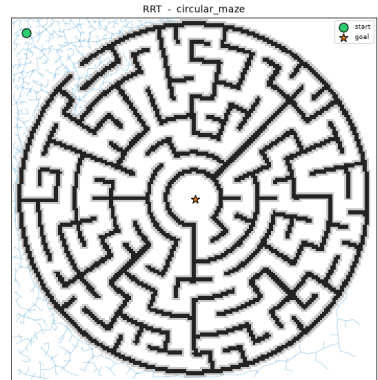


A* (esq.) · GVD (centro) · RRT (dir.). Azul: explorados (A*) / esqueleto (GVD). Vermelho: caminho final.

Resultados — *autolab*

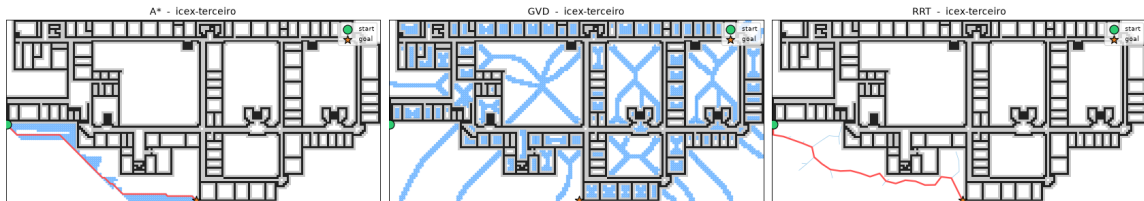
A* (esq.) · GVD (centro) · RRT (dir.).

Limitação do RRT — *circular_maze*



O RRT não converge no labirinto denso; A* e GVD resolvem.

Limitação do GVD — *icex-terceiro*

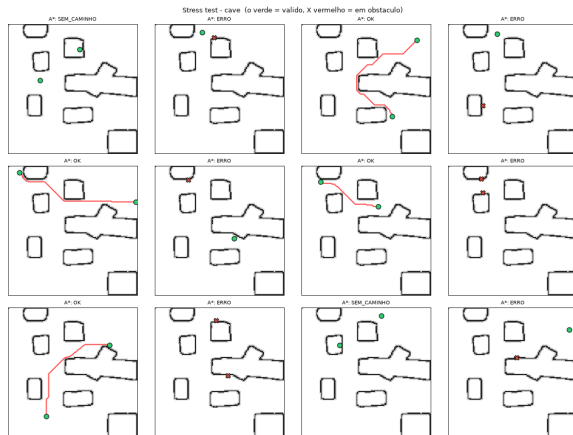


O eixo medial fragmenta nas portas estreitas e a rede não conecta *start/goal*; o A* resolve.

Teste de robustez

start/goal sorteados aleatoriamente, incluindo pontos em obstáculos:

- verde = ponto válido;
- X = ponto em obstáculo $\Rightarrow$ erro tratado (sem quebrar);
- pontos válidos desconexos $\Rightarrow$ "sem caminho".



Sumário

- 1 Introdução
- 2 Representação do Ambiente
- 3 A*
- 4 Brushfire + GVD
- 5 RRT
- 6 Simulação no CoppeliaSim
- 7 Resultados
- 8 Conclusão**

Conclusão

- **A***: caminhos ótimos e determinísticos — melhor quando a prioridade é a **distância**.
- **GVD**: caminhos mais seguros (afastados das paredes) — melhor quando a prioridade é a **segurança**.
- **RRT**: flexível e simples, ótimo em espaços abertos — mas probabilístico e fraco em corredores estreitos.
- Cada método tem uma **fraqueza demonstrada** num mapa real; onde um falha, outro resolve.
- A integração com o Coppeliasim permitiu **validar visualmente** os três planejadores.

Obrigado!

Gabriel Vaz Cançado Ferreira · Phillip Ribeiro Costa